<?php

MATERIAL DE ESTUDIO / DESARROLLO WEB

UNIDAD DIDÁCTICA 02

# Tutorial 02
# Vehículo, puertas y ventanas

Composición de objetos y responsabilidades

PHP · POO · Composición · Objetos

APUNTES DE CLASE

Desarrollo web · PHP · Tutorial guiado

{ PHP }

**UD 02 · DWES · PHP 8.1+**

# Tutorial 02 · Vehículo, puertas y ventanas

**Objetivo:** Vamos a representar un coche mediante varios objetos relacionados: cada vehículo contiene puertas y cada puerta contiene una ventana. Así se entiende la composición y se comprueba que los objetos tienen un estado independiente.

## Antes de comenzar

Utiliza PHP 8.1 o posterior y un servidor web local (por ejemplo, Apache de XAMPP). Crea una carpeta de proyecto dentro de `htdocs/UD2/` y **escribe los archivos con el contenido mostrado en cada paso**. El directorio `Codigo/` contiene esos mismos archivos listos para ejecutar.

Crea `htdocs/UD2/Vehiculo/` con `Ventana.php`, `Puerta.php`, `Vehiculo.php` e `index.php`. La URL de acceso será `http://localhost/UD2/Vehiculo/index.php`.

## Paso 1. Escribir la clase `Ventana`

Empieza creando `Ventana.php`, que es la pieza más pequeña del sistema. Su propiedad `$abierta` comienza en `false`. Los métodos `abrir()` y `cerrar()` cambian ese valor, `estaAbierta()` devuelve un booleano para consultarlo y `__toString()` lo convierte en «abierta» o «cerrada».

Ningún método imprime en pantalla. De esta forma la misma ventana se puede utilizar desde otras clases, formularios o páginas sin mezclarse con HTML.

**Archivo `Codigo/Ventana.php`**

```php
<?php
declare(strict_types=1);

class Ventana
{
    private bool $abierta = false;

    public function abrir(): void
    {
        $this->abierta = true;
    }

    public function cerrar(): void
    {
        $this->abierta = false;
    }

    public function estaAbierta(): bool
    {
        return $this->abierta;
    }

    public function __toString(): string
    {
        return $this->abierta ? 'abierta' : 'cerrada';
    }
}
```

Cuando una ventana se abre, solo cambia su propio `$abierta`. Otras instancias de `Ventana` no se alteran.

## Paso 2. Escribir la clase `Puerta`

Ahora crea `Puerta.php`. Al principio `require_once` importa la clase `Ventana`. En el constructor de la puerta se ejecuta `new Ventana()`: **cada puerta obtiene su propia ventana**. Una puerta también tiene el booleano `$abierta` y métodos para abrirla o cerrarla.

`getVentana(): Ventana` devuelve el objeto ventana que pertenece a esa puerta. Así, desde fuera, podremos llamar a `$puerta->getVentana()->abrir()`: primero obtenemos la ventana y luego modificamos su estado. `__toString()` presenta juntos el estado de la puerta y el de su ventana.

**Archivo `Codigo/Puerta.php`**

```php
<?php
declare(strict_types=1);
require_once __DIR__ . '/Ventana.php';
class Puerta
{
    private bool $abierta = false;
    private Ventana $ventana;
    public function __construct()
    {
        $this->ventana = new Ventana();
    }
    public function abrir(): void
    {
        $this->abierta = true;
    }
    public function cerrar(): void
    {
        $this->abierta = false;
    }
    public function estaAbierta(): bool
    {
        return $this->abierta;
    }
    public function getVentana(): Ventana
    {
        return $this->ventana;
    }
    public function __toString(): string
    {
        return ($this->abierta ? 'abierta' : 'cerrada') . ' · ventana ' . $this->ventana;
    }
}
```

Observa que una puerta y su ventana tienen estados separados: abrir la puerta no abre la ventana automáticamente. Más adelante `cerrarVehiculo()` coordinará las dos acciones.

## Paso 3. Construir el vehículo y sus métodos

En `Vehiculo.php` incluye `Puerta.php`. El constructor recibe el tipo de motor, una etiqueta opcional y un número de puertas. Valida que existan entre 2 y 6 puertas. El bucle `for` crea una instancia de `Puerta` en cada vuelta; por eso el coche tiene tantas puertas independientes como le indiquemos.

`getPuerta($i)` recibe la posición del array, empezando por 0. Si esa posición no existe, lanza `OutOfBoundsException`. `encenderApagar()` cambia el estado del motor; `cerrarVehiculo()` recorre todas las puertas y cierra cada puerta y su ventana. `puedeEntrarZBE()` únicamente comprueba si una etiqueta está en el array proporcionado, **sin interpretar normativa real**.

**Archivo `Codigo/Vehiculo.php`**

```
<?php
declare(strict_types=1);
require_once __DIR__ . '/Puerta.php';
class Vehiculo
{
    private array $puertas = [];
    private bool $arrancado = false;
    public function __construct(private string $motor = 'Gasolina', private ?string $etiqueta = null,
int $numeroPuertas = 4)
    {
        if ($numeroPuertas < 2 || $numeroPuertas > 6) {
            throw new InvalidArgumentException('Número de puertas entre 2 y 6');
        }
        for ($i = 0; $i < $numeroPuertas; $i++) {
            $this->puertas[] = new Puerta();
        }
    }
    public function getPuerta(int $i): Puerta
    {
        if (!isset($this->puertas[$i])) {
            throw new OutOfBoundsException('Puerta no encontrada');
        }
        return $this->puertas[$i];
    }
    public function encenderApagar(): bool
    {
        $this->arrancado = !$this->arrancado;
        return $this->arrancado;
    }
    public function cerrarVehiculo(): void
    {
        foreach ($this->puertas as $p) {
            $p->cerrar();
            $p->getVentana()->cerrar();
        }
    }
    public function puedeEntrarZBE(array $etiquetasPermitidas): bool
    {
        return $this->etiqueta !== null && in_array($this->etiqueta, $etiquetasPermitidas, true);
    }
    public function __toString(): string
    {
        $partes = [];
        foreach ($this->puertas as $i => $p) {
            $partes[] = 'Puerta ' . ($i + 1) . ': ' . $p;
        }
        return $this->motor . ' / etiqueta ' . ($this->etiqueta ?? 'sin etiqueta') . ' / ' . implode('
| ', $partes);
    }
}
```

No se utiliza `array_fill(0, 4, new Puerta())`: esa construcción colocaría repetidamente la **misma instancia** de puerta. El bucle `for` sí construye cuatro objetos diferentes.

## Paso 4. Preparar la demostración en `index.php`

En `index.php` se crea un coche de cuatro puertas. Se abre únicamente la primera puerta y su ventana. Se muestran los estados de todas las puertas, se consulta la segunda ventana y se llama a `cerrarVehiculo()`. También se alterna el motor, se comprueba una lista de etiquetas de ejemplo y se solicita una puerta que no existe para mostrar cómo se captura la excepción.

Todo el HTML se imprime desde este archivo: las clases se han limitado a gestionar estados y devolver datos.

**Archivo `Codigo/index.php`**

```
<?php
declare(strict_types=1);
require_once __DIR__ . '/Vehiculo.php';

$coche = new Vehiculo('Gasolina', 'C', 4);
$coche->getPuerta(0)->abrir();
$coche->getPuerta(0)->getVentana()->abrir();

echo '<h2>Antes de cerrar</h2>';
echo htmlspecialchars((string) $coche, ENT_QUOTES, 'UTF-8') . '<br>';
echo 'Segunda ventana abierta: '
    . ($coche->getPuerta(1)->getVentana()->estaAbierta() ? 'sí' : 'no') . '<br>';

$coche->cerrarVehiculo();
echo '<h2>Después de cerrar</h2>';
echo htmlspecialchars((string) $coche, ENT_QUOTES, 'UTF-8') . '<br>';

echo '<p>Motor encendido: ' . ($coche->encenderApagar() ? 'sí' : 'no') . '</p>';
// Comprobación inventada para practicar arrays; no reproduce la normativa de ZBE.
echo '<p>Etiqueta aceptada en este ejemplo: '
    . ($coche->puedeEntrarZBE(['0', 'ECO', 'C']) ? 'sí' : 'no') . '</p>';

try {
    $coche->getPuerta(99);
} catch (OutOfBoundsException $error) {
    echo '<p>' . htmlspecialchars($error->getMessage(), ENT_QUOTES, 'UTF-8') . '</p>';
}
```

El índice 0 corresponde a la primera puerta y el 1 a la segunda. Después de abrir la primera ventana, `getPuerta(1)->getVentana()->estaAbierta()` debe seguir devolviendo `false`.

## Resultado y comprobación

Al visitar la URL aparecen «Antes de cerrar» y «Después de cerrar». En la primera parte la puerta 1 y su ventana están abiertas; la segunda ventana figura como «no». En la segunda parte todas las puertas y ventanas están cerradas. También aparece que el motor está encendido, que la etiqueta C se acepta **en esta simulación** y el mensaje «Puerta no encontrada» al pedir el índice 99.

**Lo importante:** un objeto puede contener otros objetos, y la composición permite coordinar acciones sin compartir accidentalmente el estado.